

Tài liệu thiết kế kỹ thuật

Thiết kế nền tảng, pipeline triển khai và bảo mật

Mã tài liệu	OBE-TDD-001-VI
Phiên bản	1.0
Ngày	2026-10-09
Trạng thái	Đã duyệt cho môi trường dev
Người phụ trách	khaipd18 (DevOps / Cloud)
Đối tượng đọc	Kỹ sư Cloud và DevOps, người review bảo mật, người bảo trì
Phân loại	Nội bộ
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Kiểm soát tài liệu

Lịch sử thay đổi

Phiên bản	Ngày	Thay đổi	Người thực hiện
1.0	2026-10-09	Phát hành lần đầu: hạ tầng, nền tảng Kubernetes, CI/CD, bảo mật và các quyết định thiết kế.	khaipd18

Tài liệu liên quan

- OBE-RUN-001-VI Sổ tay vận hành (docs/manuals/operations-runbook.vi.pdf)
- Bản tiếng Anh của tài liệu này: docs/manuals/technical-design.en.pdf (OBE-TDD-001)
- README.vi.md: tóm tắt dự án, kết quả kiểm chứng và hướng dẫn chạy thử local
- docs/infrastructure/: sơ đồ HLD/LLD sửa được (.drawio) và YAML spec tương ứng

Mục lục

1	Giới thiệu	3
1.1	Mục đích	3
1.2	Phạm vi	3
1.3	Hiện trạng	3
1.4	Quy ước	3
2	Tổng quan hệ thống	4
2.1	Ứng dụng	4
2.2	Repository này bổ sung những gì	4
2.3	Kiến trúc tổng thể	5
3	Yêu cầu và ràng buộc	6
3.1	Yêu cầu phi chức năng	6
3.2	Ràng buộc	6
4	Thiết kế hạ tầng AWS	7
4.1	Mạng	7
4.2	Security group	8
4.3	Compute: Amazon EKS	8
4.4	Container registry: Amazon ECR	9
4.5	Định danh và phân quyền	9
4.6	Terraform state	10
4.7	Cấu trúc Infrastructure as Code	10
5	Thiết kế nền tảng Kubernetes	11
5.1	Namespace và Pod Security	11
5.2	Helm chart dùng chung	11
5.3	Traffic giữa các service	11
5.4	Khả năng chịu lỗi	12
5.5	Observability	12
6	Thiết kế pipeline triển khai	13
6.1	Continuous integration	13
6.2	Continuous delivery	14
6.3	Pipeline hạ tầng	14
6.4	Kiểm soát trên repository	14
7	Thiết kế bảo mật	15
7.1	Biện pháp theo từng rủi ro	15
7.2	Ngoại lệ đã chấp nhận	15
7.3	Chuỗi cung ứng phần mềm	16
8	Các quyết định thiết kế	17
9	Kiểm chứng	18
10	Rủi ro, hạn chế và roadmap	19
10.1	Rủi ro đã biết	19
10.2	Roadmap	19
	Phụ lục A. Phiên bản	20
	Phụ lục B. Thuật ngữ	21

1 Giới thiệu

1.1 Mục đích

Tài liệu này mô tả cách nền tảng Online Boutique được xây dựng trên AWS (hạ tầng, nền tảng Kubernetes, pipeline triển khai, các lớp bảo mật) và lý do đằng sau từng lựa chọn thiết kế. Đây là tài liệu tham chiếu cho bất kỳ ai thay đổi nền tảng. Các quy trình hằng ngày (triển khai, ra bản mới, xử lý sự cố) nằm trong Sổ tay vận hành (OBE-RUN-001-VI).

1.2 Phạm vi

Trong phạm vi:

- Hạ tầng AWS, không gắn với một tài khoản cụ thể, ở region ap-southeast-1: VPC, EKS, ECR, VPC endpoint, IAM role cho GitHub Actions, backend lưu Terraform state.
- Nền tảng Kubernetes trong namespace dev-eks: Helm chart, cấu hình Argo CD, Pod Security, NetworkPolicy, autoscaling, bộ monitoring.
- CI/CD: các workflow GitHub Actions, quality gate và security gate, GitOps write-back, pipeline Terraform, các kiểm soát trên repository.

Ngoài phạm vi:

- Mã nguồn ứng dụng trong `src/` và `proto/`. Đây là Online Boutique của Google (Apache License 2.0) và được giữ nguyên; những phát hiện chỉ sửa được bằng cách đổi mã nguồn này thì được báo cáo, không sửa.
- Môi trường staging và production. Hiện chỉ có một môi trường (dev); Mục 10.2 liệt kê những gì môi trường production cần thêm.

1.3 Hiện trạng

Nền tảng đã chạy trên Amazon EKS vào tháng 5/2026 (Terraform dựng hạ tầng, CI đẩy image lên ECR qua OIDC, Argo CD sync các service). Sau đó môi trường AWS được gỡ để tránh chi phí và tài khoản ban đầu không còn được dùng. Các lớp bảo mật và độ sẵn sàng thêm vào sau này được kiểm chứng trên cluster kind ở máy local và bằng `terraform plan` chỉ đọc (73 resource sẽ được tạo, không lỗi). Mục 9 trình bày chi tiết.

1.4 Quy ước

- Chữ `monospace` dùng cho tên có thật trong repository hoặc trên AWS: file, resource, lệnh.
- Tên release trong cluster có dạng `<service>-dev` (ví dụ `frontend-dev`); các bảng chỉ ghi phần tên service.
- Đường link trong ngoặc vuông trỏ tới tài liệu AWS làm căn cứ cho nhận định đó.

2 Tổng quan hệ thống

2.1 Ứng dụng

Online Boutique là ứng dụng thương mại điện tử mẫu gồm 10 microservice viết bằng 5 ngôn ngữ, gọi nhau qua gRPC. Người dùng xem sản phẩm, thêm vào giỏ hàng (lưu trong Redis) rồi thanh toán; bước checkout gọi tới payment, shipping, email, currency và cart.

Bảng 1: Các service triển khai vào dev-eks (theo gitops/dev-eks/values-*.yaml)

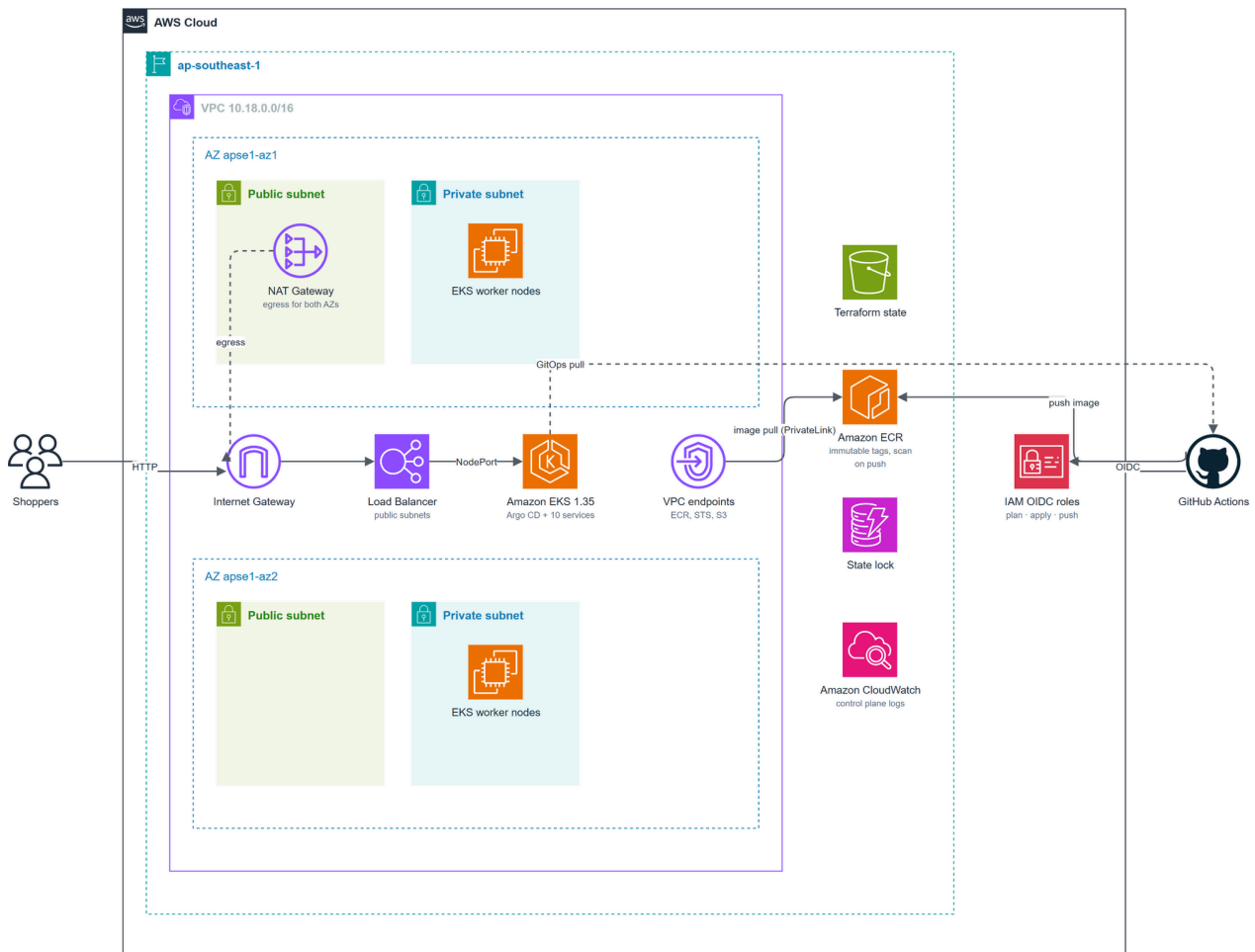
Service	Ngôn ngữ	Port	Nhận lời gọi từ	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

2.2 Repository này bổ sung những gì

Repository bổ sung toàn bộ phần bao quanh ứng dụng để chạy được trên AWS:

Lớp	Nội dung
Infrastructure as Code	Root module Terraform và 5 module (vpc, eks, ecr, vpc-endpoints, github-oidc-role).
Đóng gói	Một Helm chart dùng chung (helm-charts/) cho cả 12 release.
Continuous delivery	Argo CD ApplicationSet sinh một Application cho mỗi service từ gitops/dev-eks/.
Continuous integration	5 workflow theo ngôn ngữ, một workflow Terraform và một workflow security scan.
Bảo mật	OIDC thay cho access key, Checkov, Trivy, Pod Security Admission, NetworkPolicy.
Tài liệu	README (tiếng Anh và tiếng Việt), sơ đồ HLD/LLD, tài liệu này và sổ tay vận hành.

2.3 Kiến trúc tổng thể



Hình 1: Kiến trúc tổng thể trên AWS

Traffic từ internet đi qua Internet Gateway tới load balancer trong public subnet, rồi được chuyển tới các worker node trong private subnet [AWS]. Node truy cập ECR, STS và S3 qua VPC endpoint, phần internet còn lại đi qua một NAT Gateway. Argo CD chạy trong cluster và kéo trạng thái mong muốn từ GitHub.

3 Yêu cầu và ràng buộc

3.1 Yêu cầu phi chức năng

Lĩnh vực	Yêu cầu	Cách đáp ứng
Bảo mật	Không có AWS credential dài hạn nào trong repository hay trong GitHub secret.	IAM role cho GitHub OIDC, IRSA cho VPC CNI.
Bảo mật	Chỉ code đã review mới tới được AWS; pull request không thể thay đổi hạ tầng.	PR chỉ có role plan read-only; apply phải được duyệt qua environment production.
Bảo mật	Phát hiện hạ tầng cấu hình sai và image có lỗ hổng trước khi triển khai.	Checkov chặn trước plan/apply; Trivy quét trước khi push.
Bảo mật	Pod bị chiếm quyền không gọi được tới service mà nó vốn không gọi.	Pod hardening, PSA restricted, NetworkPolicy cho từng service.
Độ sẵn sàng	Điểm vào hệ thống chịu được việc mất một pod hoặc một node.	frontend chạy 2–4 replica, có PodDisruptionBudget và rải theo zone/node.
Vận hành	Dựng lại được trạng thái cluster chỉ từ Git.	Argo CD tự động sync, prune và self-heal.
Tính di động	Chuyển sang tài khoản AWS khác không cần sửa code.	Account ID lấy từ credentials (Terraform) và biến AWS_ACCOUNT_ID (CI).
Chi phí	Môi trường dev dựng lên và gỡ đi khi cần.	Một NAT Gateway, 2 node t3.medium, có hướng dẫn terraform destroy.

3.2 Ràng buộc

- **Không sửa src/.** Ứng dụng là code upstream. Các phát hiện lint và lỗ hổng cần sửa code thì được giữ hiển thị dưới dạng warning thay vì giấu đi.
- **Một region, một môi trường.** Mọi thứ chạy ở ap-southeast-1 dưới tên dev.
- **Repository cá nhân trên GitHub.** Ruleset của repository cá nhân không cho đưa app GitHub Actions vào danh sách bypass (chỉ repository của organization làm được), điều này ảnh hưởng tới GitOps write-back (xem Mục 10).
- **Chưa có tài khoản AWS tại thời điểm viết.** CI bỏ qua mọi bước cần AWS khi repository variable AWS_ACCOUNT_ID chưa được đặt.

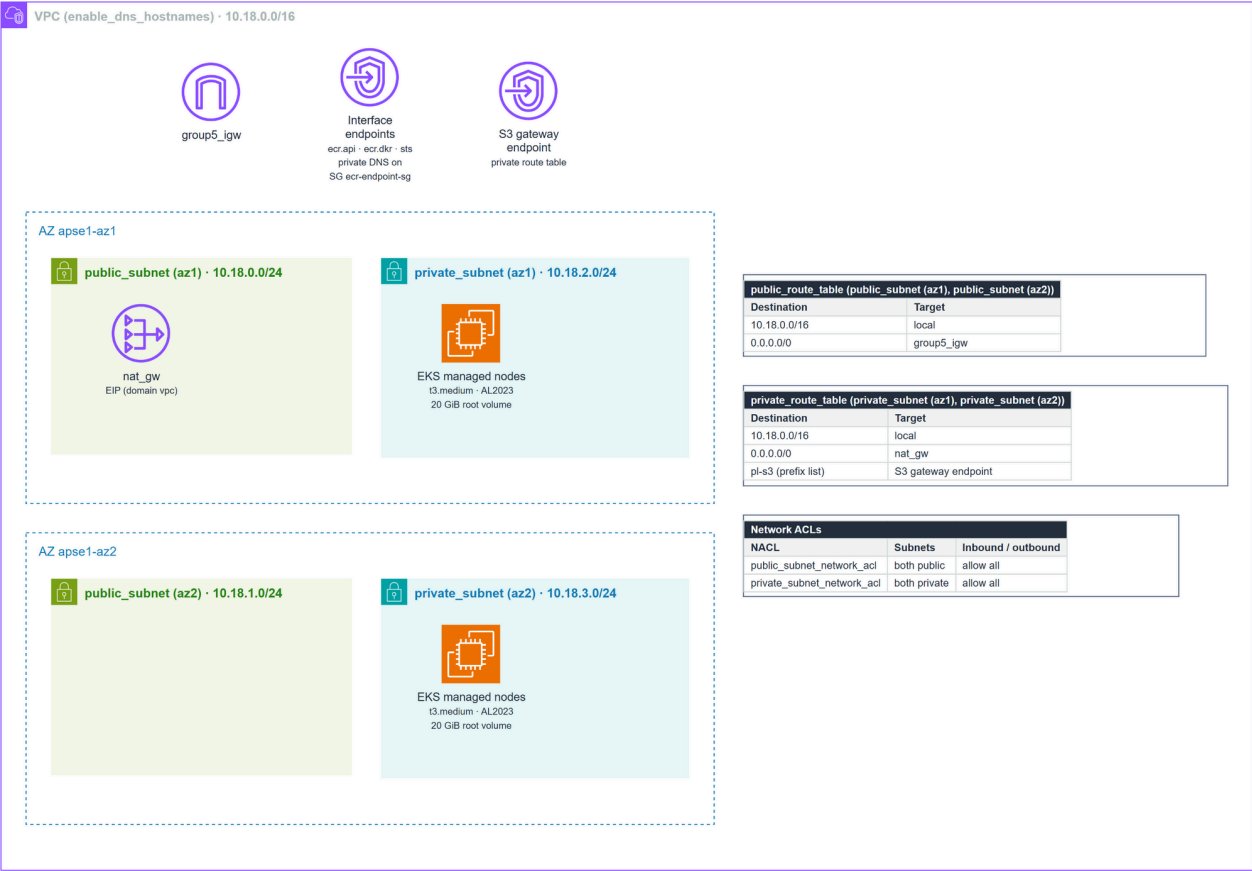
4 Thiết kế hạ tầng AWS

4.1 Mạng

Bảng 2: Quy hoạch subnet, VPC 10.18.0.0/16 (cidrsubnet(vpc_cidr, 8, n))

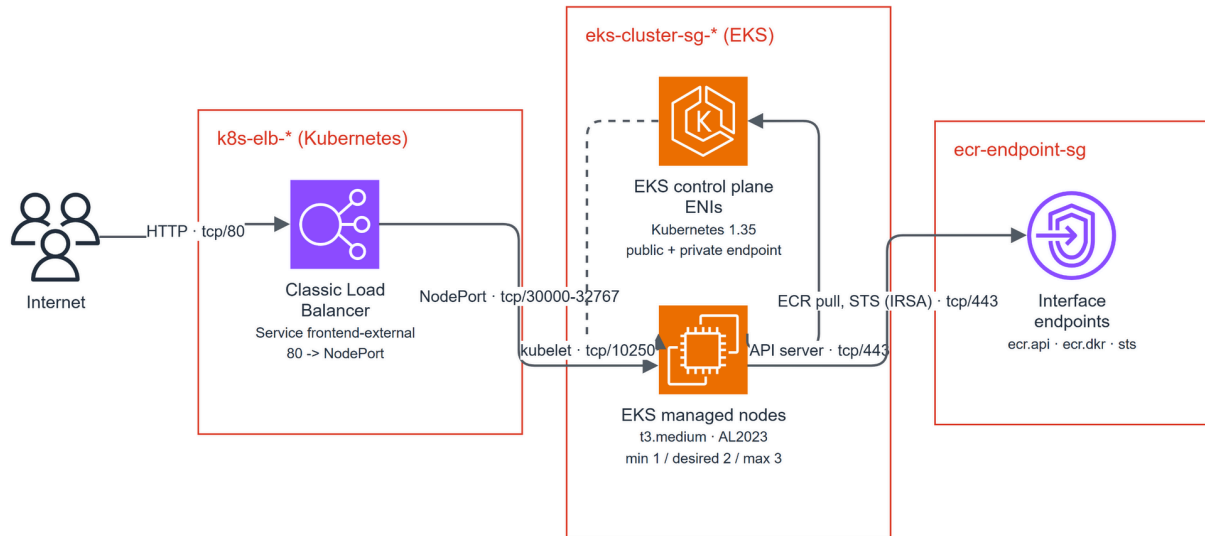
Subnet	AZ ID	CIDR	Chứa
public (az1)	apse1-az1	10.18.0.0/24	NAT Gateway, load balancer
public (az2)	apse1-az2	10.18.1.0/24	load balancer
private (az1)	apse1-az1	10.18.2.0/24	EKS nodes, interface endpoints
private (az2)	apse1-az2	10.18.3.0/24	EKS nodes, interface endpoints

- Subnet được đặt theo **AZ ID** (apse1-az1, apse1-az2) thay vì tên AZ, vì tên AZ ánh xạ tới zone vật lý khác nhau ở mỗi tài khoản.
- Route table public đưa 0.0.0.0/0 ra Internet Gateway. Route table private đưa 0.0.0.0/0 ra NAT Gateway và các prefix S3 vào S3 gateway endpoint.
- Một NAT Gateway** đặt ở public subnet az1 phục vụ cả hai AZ. Đây là quyết định tiết kiệm chi phí cho dev: nếu az1 gặp sự cố, node ở az2 mất đường ra internet (vẫn truy cập ECR/STS/S3 qua endpoint). Xem quyết định D3.
- VPC endpoint: interface endpoint ecr.api, ecr.dkr, sts có bật private DNS, và gateway endpoint cho S3 (ECR lưu image layer trên S3). Nhờ vậy việc pull image và đổi token IRSA không rời khỏi VPC [AWS].
- Network ACL cho phép mọi traffic; việc lọc do security group và NetworkPolicy của Kubernetes đảm nhận. NACL mở là finding Checkov đã chấp nhận (Mục 7.2).



Hình 2: LLD: chi tiết mạng

4.2 Security group



Security group inbound rules			
Security group	Port	Source	Description
k8s-elb-* (Kubernetes)	tcp/80	0.0.0.0/0	frontend-external listener
eks-cluster-sg-* (EKS)	all	eks-cluster-sg-* (EKS)	EKS default: nodes and control plane
eks-cluster-sg-* (EKS)	tcp/30000-32767	k8s-elb-* (Kubernetes)	NodePort, added by Kubernetes
ecr-endpoint-sg	all	10.18.0.0/16	whole VPC (only tcp/443 is needed)

Hình 3: LLD: luồng security group

- k8s-elb-* (Kubernetes tạo cho Service kiểu LoadBalancer) nhận tcp/80 từ internet.
- Security group của EKS cluster cho phép mọi traffic giữa node và control plane, cộng dải NodePort tcp/30000-32767 từ load balancer (Kubernetes tự thêm).
- ecr-endpoint-sg hiện nhận mọi giao thức từ CIDR của VPC; thực tế chỉ cần tcp/443. Việc thu hẹp nằm trong roadmap.

4.3 Compute: Amazon EKS

Thiết lập	Giá trị
Cluster	khaipd18-eks-cluster, Kubernetes 1.35
API endpoint	Public và private. Public mở cho 0.0.0.0/0 (chấp nhận cho dev, xem Mục 7.2) [AWS]
Control plane log	Đủ 5 loại: api, audit, authenticator, controllerManager, scheduler
Node group	Managed, t3.medium, AL2023_x86_64_STANDARD, on-demand, đĩa 20 GiB, min 1 / desired 2 / max 3, chỉ nằm trong private subnet
Add-on	VPC CNI bật enableNetworkPolicy và có IRSA role riêng, CoreDNS, kube-proxy, Metrics Server (community add-on)
Quyền vào cluster	Không có khối access_config: áp dụng mặc định của API (authentication mode CONFIG_MAP, người tạo cluster có quyền admin) [AWS]

Kubernetes 1.35 hết standard support ngày 27/03/2027 và hết extended support ngày 27/03/2028 [\[AWS\]](#). Quy trình nâng phiên bản nằm trong sổ tay vận hành.

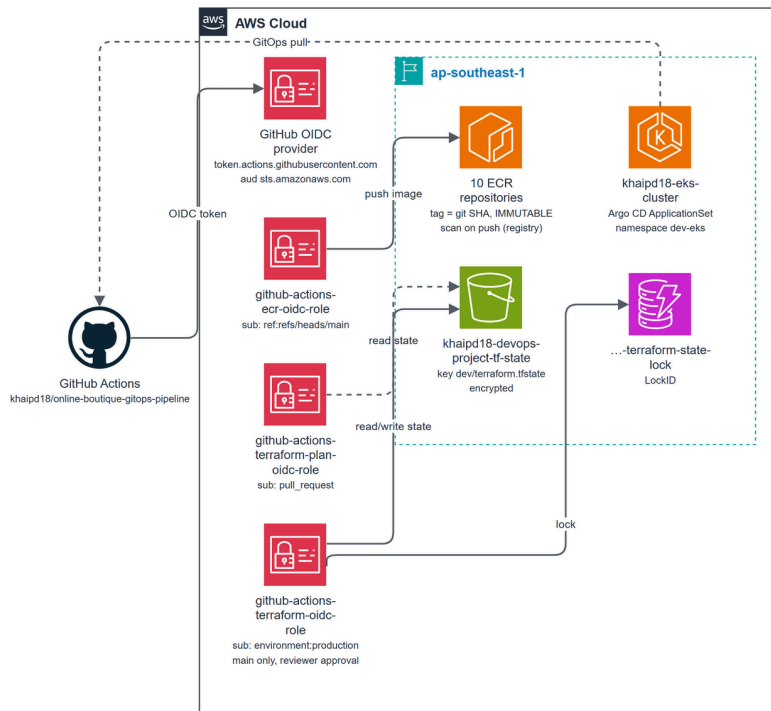
GHI CHÚ

Vì không đặt access_config, ban đầu chỉ IAM principal đã tạo cluster (người chạy terraform apply lần đầu) dùng được kubectl. Bước nên làm tiếp là đặt authentication_mode = "API_AND_CONFIG_MAP" và quản lý access entry bằng Terraform [\[AWS\]](#).

4.4 Container registry: Amazon ECR

- 10 repository, mỗi service một repository, tag IMMUTABLE: một tag image (git SHA) không bao giờ bị ghi đè [AWS].
- Lifecycle policy: image untagged hết hạn sau 14 ngày; image không được pull trong 90 ngày chuyển sang lớp lưu trữ archive.
- Scan on push (basic scanning) được cấu hình ở cấp **registry** bằng `aws_ecr_registry_scanning_configuration`, theo khuyến nghị của AWS thay cho thiết lập cấp repository đã deprecated [AWS].
- `force_delete = true` cho phép terraform destroy xóa repository còn image. Phù hợp với môi trường dev dùng xong bỏ; nên tắt khi lên production.

4.5 Định danh và phân quyền



GitHub Actions roles			
Role	Trusted sub (repo:<repo>:...)	Permissions	Used by
github-actions-ecr-oidc-role	ref:refs/heads/main	ECR push/pull on the 10 repositories	5 service CI workflows
github-actions-terraform-plan-oidc-role	pull_request	ReadOnlyAccess + state read (plan-lock=false)	terraform.yaml on pull requests
github-actions-terraform-oidc-role	environment:production	AdministratorAccess + state read/write/lock	terraform.yaml apply job, push to main + approval

Hình 4: LLD: CI/CD và IAM OIDC

GitHub Actions xác thực bằng token ngắn hạn do GitHub OIDC provider `token.actions.githubusercontent.com` cấp. Mỗi role chỉ tin một giá trị claim sub cụ thể [AWS]:

Bảng 3: Các role cho GitHub Actions (terraform/main.tf)

Role	sub được tin (sau repo: <repo>:)	Quyền
github-actions-ecr-oidc-role	ref:refs/heads/main	GitHubActions-ECR-Push-Policy (push/pull on the 10 repositories)
github-actions-terraform-plan-oidc-role	pull_request	ReadOnlyAccess + GitHubActions-Terraform-State-Read-Policy
github-actions-terraform-oidc-role	environment:production	AdministratorAccess + GitHubActions-Terraform-State-Policy

- Tin cả hai định dạng sub của GitHub: định dạng cũ `owner/repo` và định dạng immutable `owner@id/repo@id` mà GitHub dùng cho repository đổi tên sau 2026-07-15.
- Role apply tin **environment** production thay vì nhánh main. Environment này chỉ nhận deployment từ main và chờ người duyệt, đúng khuyến nghị của AWS khi trust policy dựa vào GitHub environment [\[AWS\]](#).
- GitHub không cấp OIDC token cho pull request từ fork, nên role plan chỉ được dùng bởi người có quyền ghi vào repository.
- Trong cluster, VPC CNI dùng IRSA (role gắn với kube-system/aws-node) thay vì role của node [\[AWS\]](#). Không pod ứng dụng nào cần quyền AWS.

4.6 Terraform state

- State: S3 bucket `khaipd18-devops-project-tf-state`, key `dev/terraform.tfstate`, có mã hóa, bật versioning.
- Khóa state: bảng DynamoDB `khaipd18-devops-project-terraform-state-lock` (LockID). Role plan không có quyền ghi vào bảng này nên plan cho PR chạy với `-lock=false`.
- Bucket và bảng được tạo tay một lần (bootstrap), không do Terraform quản lý.

4.7 Cấu trúc Infrastructure as Code

Module	Tạo ra
vpc	VPC, 2 public + 2 private subnet, Internet Gateway, NAT Gateway, route table, NACL (module con subnet, igw, nat_gw, route-table)
eks	Cluster, managed node group, 4 add-on, OIDC provider và IRSA role cho VPC CNI
ecr	10 repository, lifecycle policy, cấu hình scanning cấp registry
vpc-endpoints	Interface endpoint <code>ecr.api</code> , <code>ecr.dkr</code> , <code>sts</code> ; S3 gateway endpoint; <code>ecr-endpoint-sg</code>
github-oidc-role	IAM role có trust policy dựng từ danh sách repository và các giá trị sub được phép

Root module nối các module lại với nhau và khai báo GitHub OIDC provider, ba role cùng policy của chúng. Account ID không bao giờ bị hardcode: Terraform lấy từ credentials đang dùng, còn workflow đọc từ repository variable `AWS_ACCOUNT_ID`.

5 Thiết kế nền tảng Kubernetes

5.1 Namespace và Pod Security

- Mọi workload chạy trong dev-eks. Namespace do một Argo CD Application riêng (namespaces) quản lý, có Delete=false nên không lần sync nào xóa được nó.
- Pod Security Admission áp profile restricted ở chế độ enforce. Pod không có security context hợp lệ bị từ chối ngay lúc admission.
- Monitoring chạy trong monitoring; Argo CD trong argocd.

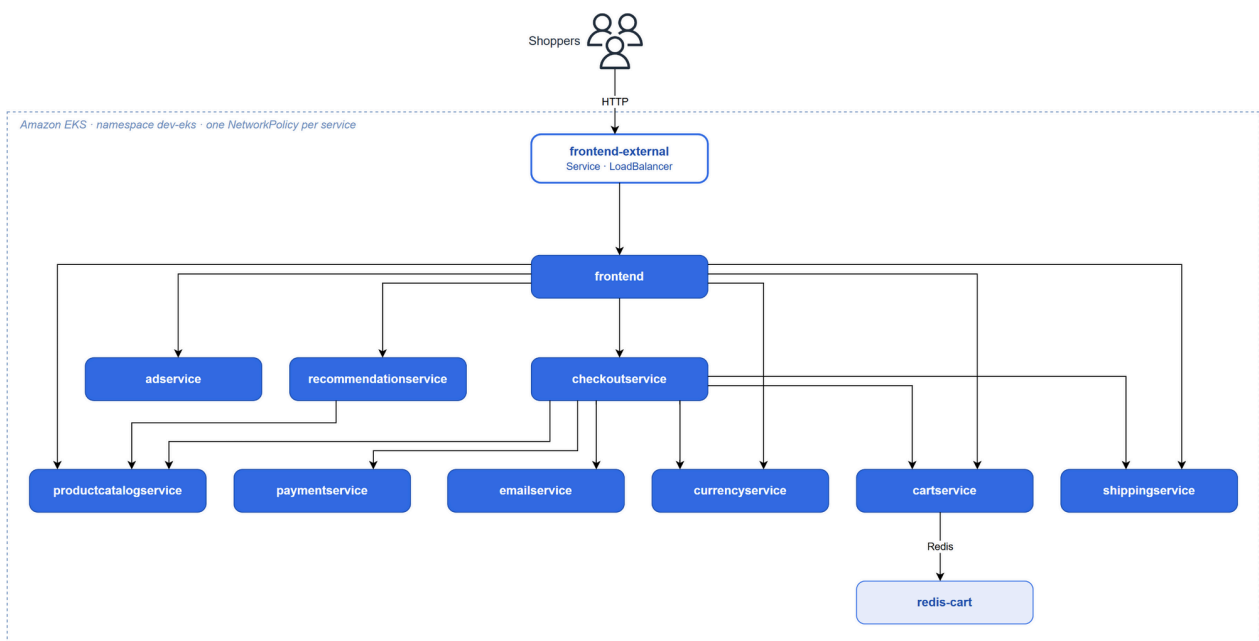
5.2 Helm chart dùng chung

Cả 12 release dùng helm-charts/ (standard-microservice). File values của từng service chỉ khai báo phần khác biệt. Chart mặc định đã an toàn:

Value	Mặc định và ý nghĩa
podSecurityContext	runAsNonRoot, UID/GID 10001, seccompProfile: RuntimeDefault
securityContext	Không leo thang đặc quyền, không privileged, root filesystem chỉ đọc, drop ALL capabilities
automountServiceAccountToken	false: không service nào gọi Kubernetes API
networkPolicy.allowFrom	Tên release được phép kết nối; để trống nghĩa là chặn mọi ingress
networkPolicy.allowFromAnywhere	Mở port của service cho mọi nguồn (dùng cho frontend)
autoscaling.*	HPA theo CPU, mặc định tắt; min 2, max 4, mục tiêu 70 %
podDisruptionBudget.*	minAvailable: 1, chỉ sinh ra khi service chạy từ 2 replica trở lên
topologySpread.enabled	Rải mềm theo zone rồi theo node (ScheduleAnyway)
service.selectorOverride	Cho Service trở vào pod của release khác (dùng cho frontend-external)

frontend-external là release có deployment.enabled: false, chỉ tạo Service kiểu LoadBalancer (80 → 8080) đứng trước các pod frontend-dev. Nhờ vậy việc mở ra internet tách biệt khỏi workload.

5.3 Traffic giữa các service



Hình 5: Các lời gọi trong cluster được NetworkPolicy cho phép

Mỗi release có một NetworkPolicy **ingress** chỉ cho phép các caller liệt kê trong `allowFrom`, và chỉ trên port của service. VPC CNI thực thi các policy này [\[AWS\]](#). Egress không bị giới hạn.

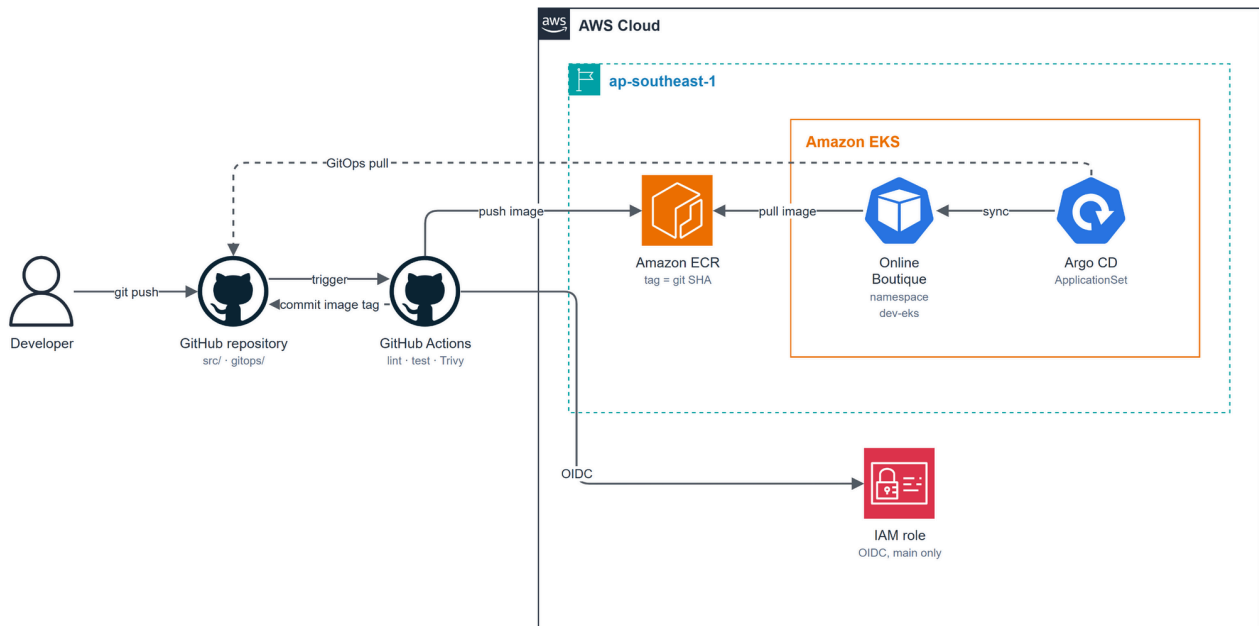
5.4 Khả năng chịu lỗi

- `frontend` (điểm vào duy nhất) có HPA theo CPU, từ 2 đến 4 replica. Metrics lấy từ add-on Metrics Server, thứ mà EKS không cài sẵn [\[AWS\]](#).
- `PodDisruptionBudget` giữ ít nhất một pod `frontend` khi drain node và khi nâng cấp.
- Các replica được rải qua zone rồi tới node khi scheduler đặt được.
- Các service còn lại chạy 1 replica. `redis-cart` bắt buộc giữ 1: nó dùng volume `emptyDir`, nên giỏ hàng mất khi pod khởi động lại.

5.5 Observability

`kube-prometheus-stack` 84.4.0 (Prometheus, Grafana, Alertmanager, node exporter, kube-state-metrics) được Argo CD deploy vào monitoring với `ServerSideApply=true` vì CRD của nó vượt giới hạn kích thước annotation của client-side apply. Control plane log của EKS đẩy về CloudWatch Logs. Hiện chưa có alert rule riêng, dashboard cho từng service, kho log tập trung hay tracing.

6 Thiết kế pipeline triển khai



Hình 6: Từ commit tới cluster

6.1 Continuous integration

Mỗi ngôn ngữ có workflow riêng, chạy khi có push lên main vào đường dẫn `src/` của ngôn ngữ đó, hoặc khi chạy tay. `dorny/paths-filter` dựng matrix gồm các service vừa thay đổi, và mỗi service chạy:

1. Lint, unit test và dependency scan (Bảng 4).
2. Đăng nhập AWS bằng role ECR, rồi kiểm tra image của commit này đã có chưa (tag là immutable nên không bao giờ build lại).
3. Build image và quét bằng Trivy (composite action `.github/actions/trivy-scan`): finding HIGH/CRITICAL đẩy lên GitHub code scanning (SARIF), SBOM CycloneDX lưu thành artifact.
4. Push image lên ECR, tag bằng git SHA.
5. Ghi `image.repository` và `image.tag` vào `gitops/dev-eks/values-<service>.yaml` rồi push commit có `[skip ci]` (có vòng retry khi nhiều service chạy cùng lúc).

Khi `AWS_ACCOUNT_ID` chưa được đặt, bước 2, 4 và 5 bị bỏ qua; image được build với tên `local/<service>:<sha>` và vẫn được quét.

Bảng 4: Quality gate

Stack	Lint / format	Test	Dependency scan
Go (4 services)	golangci-lint v1.64.8 (blocking)	go test (blocking)	govulncheck v1.8.0 (warning)
.NET (cartservice)	dotnet format (warning)	dotnet test (blocking)	dotnet list package --vulnerable (blocking)
Java (adservice)	google-java-format (warning)	gradle test (blocking)	–
Node.js (2 services)	ESLint 8 (blocking)	–	npm audit (warning)
Python (2 services)	flake8 (blocking)	pytest (blocking when tests exist)	bandit (blocking)

Các bước ghi **warning** chỉ sửa được bằng cách đổi code upstream. Chúng dùng `continue-on-error` để hiện thành warning trên mỗi lần chạy thay vì bị giấu đi. Trivy chạy ở chế độ báo cáo (`blocking: 'false'`) cũng vì lý do này.

6.2 Continuous delivery

Một ApplicationSet (`microservices-dev-eks`) sinh 12 Application tên `<service>-dev` từ list generator. Mỗi Application render `helm-charts/` với `gitops/dev-eks/values-<service>.yaml`, đích là namespace `dev-eks`, và tự động sync với `prune` và `selfHeal`. Mọi thay đổi tay trên cluster đều bị đưa về đúng như trong Git. Thêm một service nghĩa là thêm một file values và một phần tử trong list.

6.3 Pipeline hạ tầng

`terraform.yaml` chạy cho pull request và push có thay đổi trong `terraform/` hoặc chính file workflow:

Job	Hành vi
Checkov Scan	Luôn chạy. Blocking: finding nào không có trong <code>terraform/.checkov.baseline</code> sẽ làm run fail.
Terraform Plan	Chỉ cho pull request từ chính repository này. Role read-only, <code>fmt -check, validate, plan -lock=false</code> .
Terraform Apply	Chỉ khi push lên main. Dùng environment <code>production</code> , nên phải chờ người duyệt rồi mới nhận được OIDC token; sau đó <code>apply -auto-approve</code> .

Plan và Apply bị bỏ qua khi `AWS_ACCOUNT_ID` chưa được đặt. Lần apply đầu tiên ở một tài khoản mới phải chạy từ máy local, vì các role cho GitHub lúc đó chưa tồn tại.

6.4 Kiểm soát trên repository

- Ruleset `protect-main`: cấm xóa nhánh, cấm force push, bắt buộc qua pull request; admin của repository được bypass. Không đặt status check bắt buộc, vì workflow nào cũng lọc theo path và một check bắt buộc không bao giờ chạy sẽ chặn pull request mãi.
- Environment `production`: chỉ nhận deployment từ nhánh main, người duyệt bắt buộc là `khaipd18`.
- `GITHUB_TOKEN` mặc định contents: `read`; job nào cần thêm mới xin thêm (`id-token: write, security-events: write, contents: write` cho bước `write-back`).

7 Thiết kế bảo mật

7.1 Biện pháp theo từng rủi ro

Rủi ro	Biện pháp	Ở đâu
Lộ access key	OIDC cho GitHub Actions, IRSA trong cluster; không lưu key ở đâu cả	terraform/main.tf, modules/eks/iam.tf
Một nhánh hay PR chiếm quyền admin AWS	Role apply chỉ tin environment production (main + người duyệt); role ECR chỉ tin main; PR chỉ có plan read-only	modules/github-oidc-role
Terraform cấu hình sai	Checkov chặn trước plan/apply; finding đã review nằm trong baseline	.checkov.yaml, terraform/.checkov.baseline
Image có lỗ hổng	Trivy quét trước khi push, kết quả lên code scanning, mỗi image có SBOM	.github/actions/trivy-scan
Image bị ghi đè	Tag immutable, tag = git SHA	modules/ecr
Container bị chiếm quyền	Non-root, root FS chỉ đọc, drop ALL, seccomp, không mount service account token	helm-charts/values.yaml
Pod không đạt chuẩn	PSA restricted ở chế độ enforce	gitops/namespaces/dev-eks.yaml
Lateral movement	NetworkPolicy ingress cho từng service theo đúng luồng gọi gRPC	helm-charts/templates/networkpolicy.yaml
Token CI thừa quyền	Mặc định contents: read	.github/workflows/*
Thay đổi chưa review trên main	Ruleset protect-main	Cài đặt GitHub

7.2 Ngoại lệ đã chấp nhận

Các finding Terraform đã review và giữ trong terraform/.checkov.baseline (tổng 12):

Check	Finding	Lý do / kế hoạch
CKV_AWS_39	EKS bật public endpoint	Cần cho kubectl và CI khi không có VPN; giới hạn CIDR hoặc chuyển private (roadmap)
CKV_AWS_58	Chưa dùng KMS key tự quản lý cho secret của EKS	EKS 1.28+ đã mặc định mã hóa envelope toàn bộ dữ liệu Kubernetes API bằng key do AWS sở hữu [AWS] thêm customer managed key nếu cần tự kiểm soát hay audit key
CKV_AWS_229-232	NACL cho phép port 20, 21, 22, 3389 (x2 NACL)	Việc lọc do security group và NetworkPolicy đảm nhận
CKV2_AWS_11	Tắt VPC flow log	Chi phí; bật khi lên production
CKV2_AWS_12	Default security group chưa bị giới hạn	Không resource nào dùng; giới hạn khi lên production

Các check bị bỏ qua toàn cục trong .checkov.yaml: CKV_AWS_163 (scan on push đặt ở cấp registry, check này không nhìn thấy), CKV_AWS_136 (ECR mặc định mã hóa AES-256), CKV2_AWS_1 (báo nhầm khi dùng aws_network_acl_association).

Manifest Kubernetes render từ chart còn 12 finding Checkov (từ 128 trước khi hardening): image chưa pin theo digest (11) và pull policy của Redis (1). GitHub Actions có 5 finding ở input tự do custom_tag của workflow_dispatch.

7.3 Chuỗi cung ứng phần mềm

- Image được quét trước khi push; số lỗi hỏng HIGH/CRITICAL đã có bản vá được đếm cho từng image ở mỗi lần chạy và hiện trong tab **Security** của GitHub.
- Phiên bản công cụ được pin (Checkov, Trivy, golangci-lint, govulncheck, Terraform, AWS provider). GitHub Actions được pin theo major version, chưa pin theo commit SHA.
- Image chưa được ký và chưa pin theo digest (roadmap).

8 Các quyết định thiết kế

ID	Quyết định	Lý do	Xem xét lại khi
D1	Dùng IAM role cho GitHub OIDC thay access key	Không có secret để lộ hay phải xoay vòng; có thể giới hạn tin cậy theo nhánh, PR hoặc environment.	–
D2	Apply phải qua GitHub environment	Chỉ merge thôi thì không đổi được hạ tầng; bước duyệt hiện trên giao diện GitHub và có lịch sử audit.	Có thêm người duyệt (bật “prevent self-review”).
D3	Một NAT Gateway cho cả hai AZ	NAT Gateway tính tiền theo giờ và theo GB [AWS] chấp nhận single point of failure ở dev.	Production: dùng regional NAT gateway, tự trải qua các AZ và không cần public subnet [AWS] .
D4	Một Helm chart dùng chung	Thiết lập bảo mật mặc định khai báo một lần; thêm service chỉ cần một file values.	Có service cần resource mà chart không biểu diễn được.
D5	ApplicationSet với list generator	Danh sách những gì được deploy rõ ràng, review trong Git.	Có nhiều môi trường (chuyển sang matrix hoặc git generator).
D6	Tag image = git SHA, tag ECR immutable	Mỗi image đang chạy ứng với đúng một commit; rollback = SHA trước đó.	–
D7	Classic Load Balancer từ Service kiểu LoadBalancer	Không phải cài thêm controller cho dev.	Cần TLS, định tuyến theo path hoặc WAF (AWS Load Balancer Controller).
D8	IRSA cho VPC CNI	Chạy trên mọi phiên bản EKS và có từ đầu.	EKS Pod Identity giờ là cách đơn giản hơn: không cần OIDC provider cho từng cluster, một trust principal duy nhất [AWS] .
D9	Khóa state bằng DynamoDB	Mọi phiên bản Terraform dùng lúc bắt đầu dự án đều hỗ trợ.	Chuyển sang khóa trực tiếp trên S3 (roadmap).
D10	Finding của code upstream để ở mức warning	Code ứng dụng không được bảo trì ở repository này.	Dependency được nâng cấp (khi đó chuyển gate sang blocking).

9 Kiểm chứng

Kiểm tra	Kết quả
Chạy trên Amazon EKS (05/2026)	Terraform dựng hạ tầng, image push qua OIDC, Argo CD sync các service. Bằng chứng: commit của bot 14bc93f, eb86a9a, 46e66d5.
End-to-end trên kind (Kubernetes 1.37)	11/11 pod ready với security context của chart; xem sản phẩm, giỏ hàng, đổi tiền tệ và checkout đều chạy; PSA từ chối pod không đạt chuẩn; NetworkPolicy chặn 3/3 kết nối trái phép.
HPA và PDB trên cluster kind 3 node	frontend scale 2 → 3 ở 82 % CPU; drain node cuối cùng còn chạy frontend bị PDB chặn.
Terraform	fmt và validate pass; plan chỉ đọc trên state rỗng: 73 resource sẽ được tạo, không lỗi.
Checkov 3.3.22	Terraform 0 finding mới (12 trong baseline); Kubernetes 999 pass / 12 fail; GitHub Actions 5 fail.
CI không có AWS (08/10/2026)	Cả 5 workflow theo ngôn ngữ xanh; bước AWS được bỏ qua; kết quả Trivy được tải lên cho cả 10 image.

10 Rủi ro, hạn chế và roadmap

10.1 Rủi ro đã biết

Rủi ro	Ảnh hưởng	Giảm thiểu hiện tại
Bot CI bị ruleset của main chặn	Khi đã cấu hình AWS, lần push GitOps write-back bị từ chối và image mới không được deploy.	Chưa có: cần deploy key trong danh sách bypass hoặc write-back qua PR.
Chỉ một NAT Gateway	Mất az1 thì cả hai AZ mất đường ra internet.	Traffic tới ECR/STS/S3 đi qua endpoint.
10 service chỉ có 1 replica	Pod khởi động lại gây gián đoạn ngắn cho tính năng đó.	Kubernetes tự khởi động lại pod; frontend có từ 2 replica.
Dữ liệu giỏ hàng trong emptyDir	Giỏ hàng mất khi redis-cart khởi động lại.	Chấp nhận cho bản demo.
Lỗ hổng đã biết trong image	Một số image có finding CRITICAL đã có bản vá.	Hiện trong tab Security; có thể chuyển gate sang blocking.
EKS public endpoint mở ra internet	API server truy cập được từ mọi nơi (vẫn cần xác thực IAM).	Xác thực IAM; đã bật audit log.

10.2 Roadmap

Theo thứ tự ưu tiên:

1. Khôi phục GitOps write-back khi có ruleset trên main (deploy key trong danh sách bypass, hoặc bot mở pull request).
2. Nâng dependency của các service (Dependabot), rồi chuyển Trivy, govulncheck và npm audit sang blocking.
3. Giới hạn EKS public endpoint hoặc chuyển sang private endpoint.
4. Thu hẹp ecr-endpoint-sg về tcp/443 từ VPC.
5. Thay NAT Gateway duy nhất bằng regional NAT gateway.
6. Quản lý quyền vào cluster bằng EKS access entry trong Terraform.
7. External Secrets Operator với AWS Secrets Manager.
8. Pin image theo digest, ký bằng cosign và verify ở admission.
9. AWS Load Balancer Controller có TLS thay cho Classic Load Balancer.
10. Môi trường staging và production có luồng promote, canary bằng Argo Rollouts.
11. Alert rule theo SLO và dashboard Grafana cho từng service.
12. Khóa state trực tiếp trên S3 thay cho DynamoDB; cân nhắc EKS Pod Identity thay cho IRSA.

Phụ lục A. Phiên bản

Thành phần	Phiên bản	Khai báo ở
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml

Phụ lục B. Thuật ngữ

Thuật ngữ	Ý nghĩa
ApplicationSet	Resource của Argo CD sinh nhiều Application từ một template và một generator.
GitOps write-back	CI commit tag image mới vào Git để Argo CD deploy.
HPA / PDB	HorizontalPodAutoscaler / PodDisruptionBudget.
IRSA	IAM Roles for Service Accounts: pod assume IAM role qua OIDC provider của cluster.
OIDC	OpenID Connect. GitHub cấp một token có chữ ký cho mỗi job; AWS STS đổi token đó lấy credential tạm thời.
PSA	Pod Security Admission, admission controller có sẵn để áp Pod Security Standards.
SBOM	Software Bill of Materials (danh mục thành phần phần mềm); ở đây theo định dạng CycloneDX.
Claim sub	Subject của GitHub OIDC token, ví dụ repo:owner/repo:ref:refs/heads/main.